Resumen Comparativo:

Tipo de agente	Usa memoria	Tiene modelo del mundo	Planea	Reacciona
Reactivo simple	No	No	No	Sí
Reactivo basado en modelo	Sí	Sí	No	Sí
Deliberativo basado en objetivo	Sí	Sí	Sí	Puede

🧠 Comparación rápida:

Criterio	Búsqueda heurística (A*)	Algoritmos genéticos
Tipo de solución	Óptima o casi óptima	Aproximada, buena solución
Requiere heurística	Sí	No
Entorno	Estructurado, estático	Complejo, dinámico
Escalabilidad	Limitada	Alta (con recursos)
Tiempo de ejecución	Rápido (si heurística buena)	Lento (por generaciones)

🧬 Algoritmos genéticos

✅ **Cuándo usarlos:**

- El espacio de soluciones es **grande, complejo o mal definido**.
- No tienes una buena heurística o no puedes calcularla fácilmente.
- Buscas una **buena solución (no necesariamente óptima)** en problemas donde **la exploración global es importante**.
- El problema tiene muchas variables o restricciones no lineales.
- El entorno puede ser **dinámico o incierto.**

🧠 **Ejemplos:**

- Diseño de estructuras, circuitos o vehículos (donde hay miles de parámetros).
- Aprendizaje de estrategias en juegos.
- Optimización de funciones complejas.
- Planificación en entornos altamente dinámicos o multiobjetivo.

📌 **Ventajas:**

- Flexibilidad en problemas mal estructurados.
- Buena exploración global del espacio de soluciones.
- Puede manejar problemas no diferenciables, discontinuos o estocásticos.

📌 **Desventajas:**

- No garantiza encontrar la solución óptima.
- Necesita muchos recursos computacionales (tiempo, población, generaciones).
- A veces requiere ajuste fino de parámetros (mutación, cruce, etc.).

🔍 Búsqueda heurística (como A*)

✅ **Cuándo usarla:**

- El espacio de estados es **bien definido y finito** (como en mapas o laberintos).
- Tienes una **heurística clara y eficiente** que te guía hacia la solución.
- Necesitas una solución **óptima o casi óptima**.
- El entorno es **estático** (no cambia mientras se ejecuta el algoritmo).
- El costo de computación debe ser **moderado** y el tiempo de decisión **relativamente corto**.

🧠 **Ejemplos:**

- Planificación de rutas en mapas (GPS).
- Navegación de robots en entornos estructurados.
- Juegos de tablero (como ajedrez o puzzles).

📌 **Ventajas:**

- Preciso y eficiente si la heurística es buena.
- Garantiza la solución óptima (si la heurística es admisible).

📌 **Desventajas:**

- Escala mal con espacios de búsqueda enormes o continuos.
- Puede volverse lento si la heurística es pobre.

📌 ¿Qué es **K-means**?

K-means es uno de los algoritmos de clustering más populares.

🛠️ ¿Cómo funciona?

1. Tú eliges el número de grupos que quieres: `k` (por ejemplo, `k = 3`).
2. El algoritmo selecciona `k` puntos al azar como **centroides**.
3. Cada punto del dataset se asigna al **cluster más cercano** (según distancia euclidiana).
4. Se recalculan los centroides como el promedio de los puntos de cada grupo.
5. Repite hasta que los centroides no cambien mucho.

✅ Ventajas:

- Rápido y simple.
- Funciona bien cuando los grupos son esféricos y del mismo tamaño.

❌ Desventajas:

- Necesitas saber el número de clusters (`k`) antes.
- Puede quedar atrapado en mínimos locales.
- No maneja bien clusters con formas irregulares o densidades distintas.

📌 ¿Qué es **clustering jerárquico**?

El **clustering jerárquico** construye una **estructura en forma de árbol** (dendrograma) que muestra cómo se agrupan los datos.

🛠️ ¿Cómo funciona?

1. Cada punto empieza como su propio cluster.
2. En cada paso, se **fusionan los dos clusters más similares**.
3. Esto se repite hasta que todos los puntos están en un solo grupo.
4. Puedes **"cortar el árbol"** en cualquier nivel para obtener el número de clusters que desees.

Tipos:

- **Aglomerativo (bottom-up)**: empieza con cada punto como un cluster individual (más común).
- **Divisivo (top-down)**: empieza con todos los puntos juntos y los va dividiendo.

✅ Ventajas:

- No necesitas especificar `k` al principio (puedes decidirlo viendo el dendrograma).
- Muestra la estructura completa de agrupamiento.

❌ Desventajas:

- Es más lento que K-means en datasets grandes.
- No siempre forma clusters bien definidos si los datos son ruidosos.

🧪 ¿Cuándo usar cada uno?

Si...	Entonces...
Sabes cuántos clusters quieres	Usa **K-means**
Quieres entender cómo se agrupan los datos	Usa **Jerárquico**
Tienes pocos datos	Jerárquico va bien
Tienes muchos datos y necesitas velocidad	K-means es mejor

🧠 ¿Qué métrica debo usar?

Problema	Mejor métrica primaria
Clasificación balanceada	`accuracy`
Clasificación desbalanceada	`precision`, `recall`, `f1-score`
Regresión general	`RMSE` o `MAE` + R^2
Regresión con outliers	`MAE`
Clasificación médica	`recall` (capturar todos los casos positivos)
Clasificación de seguridad	`precision` (evitar falsos positivos)

🔍 Tabla Comparativa de Tipos de Agentes

Tipo de Agente	Características Principales	Formalismo o Ejemplo	Ventajas	Limitaciones
Reactivo Simple	- No guarda historial - Responde solo a la percepción actual	If-Else, tabla de reglas	Muy rápido y simple	No se adapta, propenso a bucles
Reactivo con Modelo	- Tiene estado interno - Usa historial para tomar decisiones	Máquina de estados finita, árboles de comportamiento	Puede manejar entornos parcialmente observables	Requiere diseño del modelo de estado
Basado en Objetivos	- Actúa para lograr una meta específica - Requiere replanificación si cambia la meta	Búsqueda, planificación	Más flexible que reactivos	Requiere buena definición de metas
Basado en Utilidad	- Evalúa múltiples opciones - Usa una función de utilidad para decidir	Función de utilidad	Toma decisiones más "óptimas"	Necesita definir funciones complejas de utilidad
Agente que Aprende	- Adquiere conocimiento con la experiencia - Mejora su comportamiento con el tiempo	Aprendizaje supervisado/no supervisado/refuerzo	Se adapta a nuevos entornos	Requiere mucha información y tiempo de entrenamiento
Deliberativo (BDI)	- Usa creencias, deseos e intenciones - Planifica de manera racional	Arquitectura BDI	Modelado realista del pensamiento humano	Complejo de implementar y mantener

🟢 Modelos explicables (caja blanca):

Fáciles de interpretar por humanos.

Modelo	¿Explicable?	¿Por qué?
Regresión lineal	✅ Sí	Puedes ver el peso (coeficiente) de cada variable.
Regresión logística	✅ Sí	Parecido a la lineal, pero para clasificación.
Árboles de decisión	✅ Sí	Puedes seguir el árbol y entender cada decisión.
K-Nearest Neighbors	🟡 Parcial	Difícil de explicar globalmente, pero se puede razonar una predicción local (vecinos).
Naive Bayes	✅ Sí	Basado en probabilidades simples y asumidas.

🔴 Modelos no explicables (caja negra):

Difíciles de entender sin herramientas externas.

Modelo	¿Explicable?	¿Por qué?
Redes neuronales profundas	❌ No	Muchas capas y parámetros ocultos; difícil de seguir.
Random Forest	❌ No (parcialmente con herramientas)	Usa muchos árboles, y no puedes seguirlos todos.
Gradient Boosting (XGBoost, etc.)	❌ No	Similar a Random Forest pero más complejo.
SVM con kernel no lineal	❌ No	Las transformaciones son difíciles de visualizar.